

Safe Change Workflow For FelsenmeAR

This guide defines how future vibe-coding changes should stay safe, decoupled, and expandable. Treat every requested change as a small product decision plus a small architecture change.

Core rule: every feature should have an owner module, explicit dependencies, a removal path, and a verification step.

Development Requirements

- Keep code loosely coupled and avoid unnecessary interdependency.
- Prefer small modules with explicit dependencies over shared global state or tightly connected functions.
- Keep src/app.js as the composition root only.
- Preserve existing behavior unless the request explicitly replaces it.
- For larger changes, refactor in small phases and commit after each verified phase.

Change Request Workflow

When asking for changes, include the desired behavior, what must stay unchanged, and whether any older behavior is now obsolete.

General change request template:

I want to change/add/remove [feature].

Desired behavior:

- ...

Keep unchanged:

- ...

Obsolete behavior to remove:

- ...

Safety requirement:

- Keep this loosely coupled.
- Do not add hidden dependencies or shared global state.
- Remove dead code if this replaces an older feature.
- Verify the app still loads and the relevant flow still works.

Implementation Rules

- Inspect the affected modules before editing.
- Keep feature logic in the smallest appropriate owner module.
- Pass dependencies explicitly instead of importing from src/app.js.
- Avoid shared global state unless it is already part of the app state model.
- Do not keep obsolete fallback branches, unused functions, stale comments, unused imports, or dead UI code.

Obsolete Code Removal

When replacing behavior, remove the old path completely. Search for the old feature name before committing so stale references do not remain.

- Remove unused functions, state fields, imports, event listeners, UI elements, CSS classes, debug hooks, comments, and documentation.

- Do not remove unrelated code.

Verification Checklist

- Run JavaScript syntax checks.
- Confirm relative module imports resolve.
- Start the local preview server.
- Smoke-test the affected UI, AR, debug, or reset flow.
- Review git diff for unrelated changes.

Prompt Templates

Adding a feature:

Add [feature].

Behavior:

- ...

Keep unchanged:

- ...

Architecture:

- Keep it loosely coupled.
- Put the feature logic in the smallest appropriate owner module.
- Use explicit dependency injection instead of importing from app.js.
- Do not add global state unless there is no safer option.

Verification:

- Run syntax checks.
- Confirm imports resolve.
- Run the local server smoke test.
- Tell me what changed and what files own the behavior.

Changing an existing feature:

Change [existing feature] so that [new behavior].

Important:

- Preserve [specific behavior].
- Replace [old behavior] if it conflicts.
- Remove obsolete code instead of leaving fallback branches behind.
- Search for stale references after the change.

Keep the code loosely coupled and commit the change separately.

Removing a feature:

Remove [feature] completely.

Please remove:

- behavior logic
- unused state fields
- unused UI elements/classes
- unused imports
- stale debug hooks
- stale docs/comments

Do not remove unrelated code. Verify that the app still loads and that nearby features still work.

Refactoring only:

Refactor [area/module] without changing behavior.

Rules:

- No feature changes.
- Keep public behavior and messages the same.
- Move code into smaller modules with explicit dependencies.
- Remove duplicated or dead code only when it is clearly unused.
- Commit after verification.

Useful Line To Include

When replacing old behavior:

Also remove anything made obsolete by this change.